

动态规划基础

把重复计算变成状态转移

Rosaya

Oasis

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

1.1 使用场景

1.1 使用场景

动态规划 (Dynamic Programming, DP) 通常用于解决一类具有大量重复子问题的最优化或计数问题。

1.1 使用场景

动态规划 (Dynamic Programming, DP) 通常用于解决一类具有大量重复子问题的最优化或计数问题。

一个问题适合 DP, 往往有以下特征:

- **状态可描述**: 当前已经处理到哪里、还剩什么限制, 可以用若干变量描述。
- **子问题会重复**: 不同决策路径可能走到同一个局面。
- **最优子结构**: 当前状态的答案可以由更小状态的答案推出。
- **转移有顺序**: 状态之间不存在无法处理的循环, 或者可以通过记忆化搜索处理。

1.1 使用场景

动态规划 (Dynamic Programming, DP) 通常用于解决一类具有大量重复子问题的最优化或计数问题。

一个问题适合 DP，往往有以下特征：

- **状态可描述**：当前已经处理到哪里、还剩什么限制，可以用若干变量描述。
- **子问题会重复**：不同决策路径可能走到同一个局面。
- **最优子结构**：当前状态的答案可以由更小状态的答案推出。
- **转移有顺序**：状态之间不存在无法处理的循环，或者可以通过记忆化搜索处理。

DP 的核心不是“套公式”，而是回答三个问题：

1. **状态是什么？**
2. **转移从哪里来？**

3. 计算顺序是什么？

1.1 使用场景

1.1 使用场景

一个非常容易理解的例子：跳格子。

有一排 n 个格子，从第 1 个格子出发，每次可以向右跳 1 格或 2 格。问走到第 n 个格子有多少种方案。

1.1 使用场景

一个非常容易理解的例子：跳格子。

有一排 n 个格子，从第 1 个格子出发，每次可以向右跳 1 格或 2 格。问走到第 n 个格子有多少种方案。

设 f_i 表示走到第 i 个格子的方案数。

如果最后一步跳 1 格，则来自 $i - 1$ ；如果最后一步跳 2 格，则来自 $i - 2$ 。

$$f_i = f_{i-1} + f_{i-2}$$

1.1 使用场景

一个非常容易理解的例子：跳格子。

有一排 n 个格子，从第 1 个格子出发，每次可以向右跳 1 格或 2 格。问走到第 n 个格子有多少种方案。

设 f_i 表示走到第 i 个格子的方案数。

如果最后一步跳 1 格，则来自 $i-1$ ；如果最后一步跳 2 格，则来自 $i-2$ 。

$$f_i = f_{i-1} + f_{i-2}$$

边界为 $f_1 = 1$, $f_2 = 1$ 或根据是否允许从起点跳到第二格定义为 $f_2 = 1$ 。如果把“站在 0 号位置”也看成一种起点，常见写法是 $f_0 = 1, f_1 = 1$ 。

1.1 使用场景

一个非常容易理解的例子：跳格子。

有一排 n 个格子，从第 1 个格子出发，每次可以向右跳 1 格或 2 格。问走到第 n 个格子有多少种方案。

设 f_i 表示走到第 i 个格子的方案数。

如果最后一步跳 1 格，则来自 $i-1$ ；如果最后一步跳 2 格，则来自 $i-2$ 。

$$f_i = f_{i-1} + f_{i-2}$$

边界为 $f_1 = 1, f_2 = 1$ 或根据是否允许从起点跳到第二格定义为 $f_2 = 1$ 。如果把“站在 0 号位置”也看成一种起点，常见写法是 $f_0 = 1, f_1 = 1$ 。

重点：状态的含义决定边界，边界不是凭感觉补的。

1.2 记忆化搜索

1.2 记忆化搜索

DP 可以从搜索开始理解。

如果我们用 DFS 求跳格子方案数，函数 $\text{dfs}(i)$ 表示走到第 i 格的方案数，那么：

$$\text{dfs}(i) = \text{dfs}(i - 1) + \text{dfs}(i - 2)$$

1.2 记忆化搜索

DP 可以从搜索开始理解。

如果我们用 DFS 求跳格子方案数，函数 $\text{dfs}(i)$ 表示走到第 i 格的方案数，那么：

$$\text{dfs}(i) = \text{dfs}(i - 1) + \text{dfs}(i - 2)$$

朴素搜索会重复计算很多次。例如 $\text{dfs}(n)$ 会调用 $\text{dfs}(n - 1)$ 和 $\text{dfs}(n - 2)$ ，而 $\text{dfs}(n - 1)$ 又会调用 $\text{dfs}(n - 2)$ 。

把已经计算过的 $\text{dfs}(i)$ 存下来，再次遇到直接返回，这就是**记忆化搜索**。

1.2 记忆化搜索

DP 可以从搜索开始理解。

如果我们用 DFS 求跳格子方案数，函数 $\text{dfs}(i)$ 表示走到第 i 格的方案数，那么：

$$\text{dfs}(i) = \text{dfs}(i - 1) + \text{dfs}(i - 2)$$

朴素搜索会重复计算很多次。例如 $\text{dfs}(n)$ 会调用 $\text{dfs}(n - 1)$ 和 $\text{dfs}(n - 2)$ ，而 $\text{dfs}(n - 1)$ 又会调用 $\text{dfs}(n - 2)$ 。

把已经计算过的 $\text{dfs}(i)$ 存下来，再次遇到直接返回，这就是**记忆化搜索**。

记忆化搜索的优点：

- 写法接近原始递归思路。
- 不需要立刻想清楚完整拓扑顺序。
- 适合从当前状态向可达状态展开的问题。

1.2 记忆化搜索

DP 可以从搜索开始理解。

如果我们用 DFS 求跳格子方案数，函数 $\text{dfs}(i)$ 表示走到第 i 格的方案数，那么：

$$\text{dfs}(i) = \text{dfs}(i - 1) + \text{dfs}(i - 2)$$

朴素搜索会重复计算很多次。例如 $\text{dfs}(n)$ 会调用 $\text{dfs}(n - 1)$ 和 $\text{dfs}(n - 2)$ ，而 $\text{dfs}(n - 1)$ 又会调用 $\text{dfs}(n - 2)$ 。

把已经计算过的 $\text{dfs}(i)$ 存下来，再次遇到直接返回，这就是**记忆化搜索**。

记忆化搜索的优点：

- 写法接近原始递归思路。
- 不需要立刻想清楚完整拓扑顺序。
- 适合从当前状态向可达状态展开的问题。

缺点：

- 递归层数可能过深。
- 常数略大。
- 有些状态顺序明确时，递推写法更清晰。

1.3 从搜索到递推

1.3 从搜索到递推

记忆化搜索和递推本质相同，只是计算顺序不同。

对于跳格子：

- 记忆化搜索：从目标 n 往前找依赖。
- 递推：从已知边界 1, 2 往后推出答案。

1.3 从搜索到递推

记忆化搜索和递推本质相同，只是计算顺序不同。

对于跳格子：

- 记忆化搜索：从目标 n 往前找依赖。
- 递推：从已知边界 $1, 2$ 往后推出答案。

对于任意 DP，都可以尝试画出状态依赖图。如果状态 u 的答案依赖状态 v ，就连一条 $v \rightarrow u$ 的边。

如果这张图是 DAG，就可以按拓扑顺序递推。

1.3 从搜索到递推

记忆化搜索和递推本质相同，只是计算顺序不同。

对于跳格子：

- 记忆化搜索：从目标 n 往前找依赖。
- 递推：从已知边界 $1, 2$ 往后推出答案。

对于任意 DP，都可以尝试画出状态依赖图。如果状态 u 的答案依赖状态 v ，就连一条 $v \rightarrow u$ 的边。

如果这张图是 DAG，就可以按拓扑顺序递推。

重点：递推顺序必须保证转移来源已经算完。

1.4 数字三角形

1.4 数字三角形

- P1216 [IOI1994] 数字三角形

给定一个数字三角形，从顶部出发，每一步只能走到下一行相邻的两个位置，求路径数字和最大值。

1.4 数字三角形

- P1216 [IOI1994] 数字三角形

给定一个数字三角形，从顶部出发，每一步只能走到下一行相邻的两个位置，求路径数字和最大值。

设 $f_{i,j}$ 表示走到第 i 行第 j 个位置时的最大和。

最后一步只能来自左上或右上：

$$f_{i,j} = a_{i,j} + \max\{f_{i-1,j-1}, f_{i-1,j}\}$$

1.4 数字三角形

- P1216 [IOI1994] 数字三角形

给定一个数字三角形，从顶部出发，每一步只能走到下一行相邻的两个位置，求路径数字和最大值。

设 $f_{i,j}$ 表示走到第 i 行第 j 个位置时的最大和。

最后一步只能来自左上或右上：

$$f_{i,j} = a_{i,j} + \max\{f_{i-1,j-1}, f_{i-1,j}\}$$

边界： $f_{1,1} = a_{1,1}$ ，越界位置视为不可达。

答案： $\max_{j=1}^n f_{n,j}$ 。

1.4 数字三角形

• P1216 [IOI1994] 数字三角形

给定一个数字三角形，从顶部出发，每一步只能走到下一行相邻的两个位置，求路径数字和最大值。

设 $f_{i,j}$ 表示走到第 i 行第 j 个位置时的最大和。

最后一步只能来自左上或右上：

$$f_{i,j} = a_{i,j} + \max\{f_{i-1,j-1}, f_{i-1,j}\}$$

边界： $f_{1,1} = a_{1,1}$ ，越界位置视为不可达。

答案： $\max_{j=1}^n f_{n,j}$ 。

这题的状态很自然，因为每条路径都必须经过某个位置。我们把“路径”压缩成了“到达这个位置的最优值”。

重点：DP 经常通过状态压缩掉已经不重要的历史。

1.5 习题

- P1002 [NOIP2002 普及组] 过河卒
- P1216 [IOI1994] 数字三角形
- P2196 [NOIP1996 提高组] 挖地雷
- CF698A Vacations

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

2.1 01 背包

2.1 01 背包

- P1048 [NOIP2005 普及组] 采药

有 n 株草药，第 i 株需要 t_i 时间，价值为 v_i 。总时间不超过 T ，每株草药最多采一次，求最大价值。

2.1 01 背包

- P1048 [NOIP2005 普及组] 采药

有 n 株草药，第 i 株需要 t_i 时间，价值为 v_i 。总时间不超过 T ，每株草药最多采一次，求最大价值。

这就是最经典的 **01 背包**。

每个物品只有两种选择：选或不选。

2.1 01 背包

- P1048 [NOIP2005 普及组] 采药

有 n 株草药，第 i 株需要 t_i 时间，价值为 v_i 。总时间不超过 T ，每株草药最多采一次，求最大价值。

这就是最经典的 **01 背包**。

每个物品只有两种选择：选或不选。

设 $f_{i,j}$ 表示只考虑前 i 个物品，总花费不超过 j 时的最大价值。

若不选第 i 个物品：

$$f_{i,j} \leftarrow f_{i-1,j}$$

若选第 i 个物品：

$$f_{i,j} \leftarrow f_{i-1,j-t_i} + v_i$$

2.1 01 背包

- P1048 [NOIP2005 普及组] 采药

有 n 株草药，第 i 株需要 t_i 时间，价值为 v_i 。总时间不超过 T ，每株草药最多采一次，求最大价值。

这就是最经典的 **01 背包**。

每个物品只有两种选择：选或不选。

设 $f_{i,j}$ 表示只考虑前 i 个物品，总花费不超过 j 时的最大价值。

若不选第 i 个物品：

$$f_{i,j} \leftarrow f_{i-1,j}$$

若选第 i 个物品：

$$f_{i,j} \leftarrow f_{i-1,j-t_i} + v_i$$

所以转移为：

$$f_{i,j} = \max\{f_{i-1,j}, f_{i-1,j-t_i} + v_i\}$$

其中第二项要求 $j \geq t_i$ 。

2.1 01 背包

2.1 01 背包

可以滚动掉第一维，用 f_j 表示当前已经处理过若干个物品后的答案。

处理第 i 个物品时：

$$f_j = \max\{f_j, f_{j-t_i} + v_i\}$$

2.1 01 背包

可以滚动掉第一维，用 f_j 表示当前已经处理过若干个物品后的答案。

处理第 i 个物品时：

$$f_j = \max\{f_j, f_{j-t_i} + v_i\}$$

01 背包一维优化时， j 必须**倒序枚举**。

因为每个物品只能用一次，倒序可以保证 f_{j-t_i} 仍然是“处理第 i 个物品之前”的状态。

2.1 01 背包

可以滚动掉第一维，用 f_j 表示当前已经处理过若干个物品后的答案。

处理第 i 个物品时：

$$f_j = \max\{f_j, f_{j-t_i} + v_i\}$$

01 背包一维优化时， j 必须**倒序枚举**。

因为每个物品只能用一次，倒序可以保证 f_{j-t_i} 仍然是“处理第 i 个物品之前”的状态。

重点：背包 DP 的枚举顺序通常由“物品能用几次”决定。

2.2 完全背包

2.2 完全背包

- P1616 疯狂的采药

有 n 种草药，第 i 种每株需要 t_i 时间，价值为 v_i ，每种可以采任意多次。总时间不超过 T ，求最大价值。

2.2 完全背包

- P1616 疯狂的采药

有 n 种草药，第 i 种每株需要 t_i 时间，价值为 v_i ，每种可以采任意多次。总时间不超过 T ，求最大价值。

这就是完全背包。

同一个物品可以选多次，所以处理第 i 个物品时，若已经选过它，还可以继续选。

2.2 完全背包

- P1616 疯狂的采药

有 n 种草药，第 i 种每株需要 t_i 时间，价值为 v_i ，每种可以采任意多次。总时间不超过 T ，求最大价值。

这就是完全背包。

同一个物品可以选多次，所以处理第 i 个物品时，若已经选过它，还可以继续选。

一维状态仍是：

$$f_j = \max\{f_j, f_{j-t_i} + v_i\}$$

但此时 j 要正序枚举。

2.2 完全背包

- P1616 疯狂的采药

有 n 种草药，第 i 种每株需要 t_i 时间，价值为 v_i ，每种可以采任意多次。总时间不超过 T ，求最大价值。

这就是完全背包。

同一个物品可以选多次，所以处理第 i 个物品时，若已经选过它，还可以继续选。

一维状态仍是：

$$f_j = \max\{f_j, f_{j-t_i} + v_i\}$$

但此时 j 要正序枚举。

正序枚举时， f_{j-t_i} 可能已经在当前物品这一轮被更新过，因此相当于允许第 i 个物品被重复使用。

重点：01 背包倒序，完全背包正序。

2.3 方案数背包

2.3 方案数背包

- P1164 小 A 点菜

有 n 道菜，第 i 道价格为 a_i ，求恰好花完 m 元的点菜方案数。每道菜最多点一次。

2.3 方案数背包

- P1164 小 A 点菜

有 n 道菜，第 i 道价格为 a_i ，求恰好花完 m 元的点菜方案数。每道菜最多点一次。

这题不是求最大价值，而是求方案数。

设 f_j 表示当前处理过的菜中，恰好花 j 元的方案数。

初始 $f_0 = 1$ ，表示什么都不选也是一种凑出 0 的方案。

2.3 方案数背包

• P1164 小 A 点菜

有 n 道菜，第 i 道价格为 a_i ，求恰好花完 m 元的点菜方案数。每道菜最多点一次。

这题不是求最大价值，而是求方案数。

设 f_j 表示当前处理过的菜中，恰好花 j 元的方案数。

初始 $f_0 = 1$ ，表示什么都不选也是一种凑出 0 的方案。

对每道菜做 01 背包转移：

$$f_j \leftarrow f_j + f_{j-a_i}$$

2.3 方案数背包

• P1164 小 A 点菜

有 n 道菜，第 i 道价格为 a_i ，求恰好花完 m 元的点菜方案数。每道菜最多点一次。

这题不是求最大价值，而是求方案数。

设 f_j 表示当前处理过的菜中，恰好花 j 元的方案数。

初始 $f_0 = 1$ ，表示什么都不选也是一种凑出 0 的方案。

对每道菜做 01 背包转移：

$$f_j \leftarrow f_j + f_{j-a_i}$$

重点：求最值时常用 max/min，求方案数时常用加法。状态含义变了，转移符号也会变。

2.4 习题

- P1048 [NOIP2005 普及组] 采药
- P1060 [NOIP2006 普及组] 开心的金明
- P1616 疯狂的采药
- P1164 小 A 点菜
- P1757 通天之分组背包
- CF189A Cut Ribbon

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

3.1 分组背包

3.1 分组背包

- P1757 通天之分组背包

有若干物品，每个物品有体积、价值和所属组。每组最多选择一个物品，背包容量为 m ，求最大价值。

3.1 分组背包

- P1757 通天之分组背包

有若干物品，每个物品有体积、价值和所属组。每组最多选择一个物品，背包容量为 m ，求最大价值。

分组背包和 01 背包的区别：

- 01 背包：每个物品独立决定选或不选。
- 分组背包：每个组内部最多选一个。

3.1 分组背包

- P1757 通天之分组背包

有若干物品，每个物品有体积、价值和所属组。每组最多选择一个物品，背包容量为 m ，求最大价值。

分组背包和 01 背包的区别：

- 01 背包：每个物品独立决定选或不选。
- 分组背包：每个组内部最多选一个。

设 f_j 表示处理完若干组后，容量为 j 的最大价值。

处理某一组时，枚举这一组中选择哪个物品：

$$f_j = \max\{f_j, f_{j-w} + v\}$$

3.1 分组背包

- P1757 通天之分组背包

有若干物品，每个物品有体积、价值和所属组。每组最多选择一个物品，背包容量为 m ，求最大价值。

分组背包和 01 背包的区别：

- 01 背包：每个物品独立决定选或不选。
- 分组背包：每个组内部最多选一个。

设 f_j 表示处理完若干组后，容量为 j 的最大价值。

处理某一组时，枚举这一组中选择哪个物品：

$$f_j = \max\{f_j, f_{j-w} + v\}$$

注意同一组内的物品不能互相转移。也就是说，处理一组时要保证所有转移都来自“处理这一组之前”的状态。

常见做法是对每组使用一个临时数组，或者容量倒序并在这一组内谨慎更新。

3.1 分组背包

- P1757 通天之分组背包

有若干物品，每个物品有体积、价值和所属组。每组最多选择一个物品，背包容量为 m ，求最大价值。

分组背包和 01 背包的区别：

- 01 背包：每个物品独立决定选或不选。
- 分组背包：每个组内部最多选一个。

设 f_j 表示处理完若干组后，容量为 j 的最大价值。

处理某一组时，枚举这一组中选择哪个物品：

$$f_j = \max\{f_j, f_{j-w} + v\}$$

注意同一组内的物品不能互相转移。也就是说，处理一组时要保证所有转移都来自“处理这一组之前”的状态。

常见做法是对每组使用一个临时数组，或者容量倒序并在这一组内谨慎更新。

重点：分组背包的循环顺序是“组 -> 容量 -> 组内物品”。

3.2 多重背包

3.2 多重背包

多重背包中，第 i 种物品最多可以选择 c_i 个，每个体积 w_i ，价值 v_i 。

直接把它拆成 c_i 个 01 物品是正确的，但当 c_i 很大时会超时。

3.2 多重背包

多重背包中，第 i 种物品最多可以选择 c_i 个，每个体积 w_i ，价值 v_i 。

直接把它拆成 c_i 个 01 物品是正确的，但当 c_i 很大时会超时。

一个经典优化是**二进制分组**。

将 c_i 拆成若干个 $1, 2, 4, 8, \dots$ 和剩余部分。例如：

$$13 = 1 + 2 + 4 + 6$$

于是最多选 13 个物品可以转化为选择若干个“大物品”，每个大物品代表若干个原物品。

3.2 多重背包

多重背包中，第 i 种物品最多可以选择 c_i 个，每个体积 w_i ，价值 v_i 。

直接把它拆成 c_i 个 01 物品是正确的，但当 c_i 很大时会超时。

一个经典优化是**二进制分组**。

将 c_i 拆成若干个 $1, 2, 4, 8, \dots$ 和剩余部分。例如：

$$13 = 1 + 2 + 4 + 6$$

于是最多选 13 个物品可以转化为选择若干个“大物品”，每个大物品代表若干个原物品。

第 k 个大物品若代表 x 个原物品，则：

- 体积为 xw_i 。
- 价值为 xv_i 。

- 只能选或不选。

于是多重背包变成若干个 01 背包物品。

3.2 多重背包

多重背包中，第 i 种物品最多可以选择 c_i 个，每个体积 w_i ，价值 v_i 。

直接把它拆成 c_i 个 01 物品是正确的，但当 c_i 很大时会超时。

一个经典优化是**二进制分组**。

将 c_i 拆成若干个 $1, 2, 4, 8, \dots$ 和剩余部分。例如：

$$13 = 1 + 2 + 4 + 6$$

于是最多选 13 个物品可以转化为选择若干个“大物品”，每个大物品代表若干个原物品。

第 k 个大物品若代表 x 个原物品，则：

- 体积为 xw_i 。
- 价值为 xv_i 。

- 只能选或不选。

于是多重背包变成若干个 01 背包物品。

为什么二进制分组可以表示任意数量？

因为 $1, 2, 4, \dots, 2^t$ 可以表示 $0 \sim 2^{t+1} - 1$ 中的所有整数，最后一个剩余块补足上界。

重点：二进制分组优化的是物品数量，而不是状态数量。

3.3 宝物筛选

3.3 宝物筛选

- P1776 宝物筛选

有 n 种宝物，每种宝物有价值、重量和数量，背包容量为 W ，求最大价值。

3.3 宝物筛选

- P1776 宝物筛选

有 n 种宝物，每种宝物有价值、重量和数量，背包容量为 W ，求最大价值。

这是多重背包模板题。

处理每种宝物时，先把数量二进制分组，再把每一组作为一个 01 背包物品加入转移。

3.3 宝物筛选

- P1776 宝物筛选

有 n 种宝物，每种宝物有价值、重量和数量，背包容量为 W ，求最大价值。

这是多重背包模板题。

处理每种宝物时，先把数量二进制分组，再把每一组作为一个 01 背包物品加入转移。

如果第 i 种宝物数量为 c_i ，拆出的组数是 $O(\log c_i)$ 。

总复杂度从直接拆分的 $O(W \sum c_i)$ 降为：

$$O\left(W \sum_{i=1}^n \log c_i\right)$$

3.4 习题

- P1757 通天之分组背包
- P1776 宝物筛选
- P1833 樱花
- P2851 [USACO06DEC] The Fewest Coins G
- P5020 [NOIP2018 提高组] 货币系统
- CF577B Modulo Sum

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

4.1 线性状态

4.1 线性状态

序列 DP 通常处理一段从左到右的数据。

常见状态：

- f_i : 以第 i 个元素结尾的最优值。
- f_i : 只考虑前 i 个元素的最优值。
- $f_{i,j}$: 处理到第 i 个位置，并且当前处于第 j 种状态。

4.1 线性状态

序列 DP 通常处理一段从左到右的数据。

常见状态：

- f_i : 以第 i 个元素结尾的最优值。
- f_i : 只考虑前 i 个元素的最优值。
- $f_{i,j}$: 处理到第 i 个位置，并且当前处于第 j 种状态。

序列 DP 的关键是判断“历史中哪些信息对未来仍有用”。

如果未来只关心当前最后一个元素，就可以用“以 i 结尾”的状态。

如果未来只关心已经处理的前缀，就可以用“前 i 个”的状态。

4.2 最大子段和

4.2 最大子段和

- P1115 最大子段和

给定一个长度为 n 的整数序列，求非空连续子段的最大和。

4.2 最大子段和

- P1115 最大子段和

给定一个长度为 n 的整数序列，求非空连续子段的最大和。

设 f_i 表示以第 i 个数结尾的最大子段和。

对于第 i 个数，有两种选择：

- 单独作为一个新子段。
- 接在以 $i-1$ 结尾的最优子段后面。

4.2 最大子段和

• P1115 最大子段和

给定一个长度为 n 的整数序列，求非空连续子段的最大和。

设 f_i 表示以第 i 个数结尾的最大子段和。

对于第 i 个数，有两种选择：

- 单独作为一个新子段。
- 接在以 $i-1$ 结尾的最优子段后面。

转移：

$$f_i = \max\{a_i, f_{i-1} + a_i\}$$

答案：

$$\max_{i=1}^n f_i$$

4.2 最大子段和

• P1115 最大子段和

给定一个长度为 n 的整数序列，求非空连续子段的最大和。

设 f_i 表示以第 i 个数结尾的最大子段和。

对于第 i 个数，有两种选择：

- 单独作为一个新子段。
- 接在以 $i-1$ 结尾的最优子段后面。

转移：

$$f_i = \max\{a_i, f_{i-1} + a_i\}$$

答案：

$$\max_{i=1}^n f_i$$

重点：“以 i 结尾”常常可以让连续性约束变得简单。

4.3 最长上升子序列

4.3 最长上升子序列

- P1020 [NOIP1999 提高组] 导弹拦截

第一问可以视为最长单调不升子序列，第二问与最长上升子序列有关。

4.3 最长上升子序列

- P1020 [NOIP1999 提高组] 导弹拦截

第一问可以视为最长单调不升子序列，第二问与最长上升子序列有关。

先看最基础的 LIS。

设 f_i 表示以第 i 个数结尾的最长上升子序列长度。

若 $j < i$ 且 $a_j < a_i$ ，那么可以从 j 转移到 i ：

$$f_i = \max\{f_i, f_j + 1\}$$

4.3 最长上升子序列

- P1020 [NOIP1999 提高组] 导弹拦截

第一问可以视为最长单调不升子序列，第二问与最长上升子序列有关。

先看最基础的 LIS。

设 f_i 表示以第 i 个数结尾的最长上升子序列长度。

若 $j < i$ 且 $a_j < a_i$ ，那么可以从 j 转移到 i ：

$$f_i = \max\{f_i, f_j + 1\}$$

朴素复杂度 $O(n^2)$ 。

这个状态很容易理解：最后一个数固定为 a_i ，那么前一个数只能从 i 前面且值更小的位置来。

4.3 最长上升子序列

- P1020 [NOIP1999 提高组] 导弹拦截

第一问可以视为最长单调不升子序列，第二问与最长上升子序列有关。

先看最基础的 LIS。

设 f_i 表示以第 i 个数结尾的最长上升子序列长度。

若 $j < i$ 且 $a_j < a_i$ ，那么可以从 j 转移到 i ：

$$f_i = \max\{f_i, f_j + 1\}$$

朴素复杂度 $O(n^2)$ 。

这个状态很容易理解：最后一个数固定为 a_i ，那么前一个数只能从 i 前面且值更小的位置来。

LIS 可以进一步用二分优化到 $O(n \log n)$, 但这属于之后的内容。现在要先理解 $O(n^2)$ 的状态与转移。

4.4 多状态序列 DP

4.4 多状态序列 DP

- CF698A Vacations

每天可能可以健身、比赛或都可以，也可以休息。不能连续两天做同一种活动，求最少休息天数。

4.4 多状态序列 DP

- CF698A Vacations

每天可能可以健身、比赛或都可以，也可以休息。不能连续两天做同一种活动，求最少休息天数。

设 $f_{i,0}$ 表示第 i 天休息时，前 i 天的最少休息天数。

$f_{i,1}$ 表示第 i 天比赛， $f_{i,2}$ 表示第 i 天健身。

4.4 多状态序列 DP

• CF698A Vacations

每天可能可以健身、比赛或都可以，也可以休息。不能连续两天做同一种活动，求最少休息天数。

设 $f_{i,0}$ 表示第 i 天休息时，前 i 天的最少休息天数。

$f_{i,1}$ 表示第 i 天比赛， $f_{i,2}$ 表示第 i 天健身。

第 i 天的状态只依赖第 $i-1$ 天的状态，并且不能与前一天活动相同。

这类题的重点是**把限制放进状态里**。

4.4 多状态序列 DP

• CF698A Vacations

每天可能可以健身、比赛或都可以，也可以休息。不能连续两天做同一种活动，求最少休息天数。

设 $f_{i,0}$ 表示第 i 天休息时，前 i 天的最少休息天数。

$f_{i,1}$ 表示第 i 天比赛， $f_{i,2}$ 表示第 i 天健身。

第 i 天的状态只依赖第 $i-1$ 天的状态，并且不能与前一天活动相同。

这类题的重点是**把限制放进状态里**。

如果只设 f_i 表示前 i 天最少休息次数，就不知道第 i 天做了什么，无法判断第 $i+1$ 天能不能继续同一种活动。

4.5 习题

- P1115 最大子段和
- P1020 [NOIP1999 提高组] 导弹拦截
- P1091 [NOIP2004 提高组] 合唱队形
- P2758 编辑距离
- CF455A Boredom
- CF698A Vacations

目录

1. 动态规划简介	3	3.2 多重背包	27
1.1 使用场景	5	3.3 宝物筛选	29
1.2 记忆化搜索	8	3.4 习题	30
1.3 从搜索到递推	10	4. 序列 DP	31
1.4 数字三角形	11	4.1 线性状态	33
1.5 习题	13	4.2 最大子段和	34
2. 背包 DP	14	4.3 最长上升子序列	36
2.1 01 背包	16	4.4 多状态序列 DP	38
2.2 完全背包	19	4.5 习题	39
2.3 方案数背包	21	5. 区间 DP	40
2.4 习题	22	5.1 思想	42
3. 分组与多重背包	23	5.2 石子合并	44
3.1 分组背包	25	5.3 能量项链	46

5.4 删除类区间 DP	47
5.5 习题	48

5.1 思想

5.1 思想

区间 DP 通常处理“合并一段区间”或“删除一段区间”的问题。

常见状态：

$$f_{l,r}$$

表示只考虑区间 $[l, r]$ 的答案。

5.1 思想

区间 DP 通常处理“合并一段区间”或“删除一段区间”的问题。

常见状态：

$$f_{l,r}$$

表示只考虑区间 $[l, r]$ 的答案。

区间 DP 的典型转移是枚举分割点：

$$f_{l,r} = \min_{l \leq k < r} \{ f_{l,k} + f_{k+1,r} + \text{cost}(l, r) \}$$

5.1 思想

区间 DP 通常处理“合并一段区间”或“删除一段区间”的问题。

常见状态：

$$f_{l,r}$$

表示只考虑区间 $[l, r]$ 的答案。

区间 DP 的典型转移是枚举分割点：

$$f_{l,r} = \min_{l \leq k < r} \{f_{l,k} + f_{k+1,r} + \text{cost}(l, r)\}$$

计算顺序一般按区间长度从小到大：

- 长度为 1 的区间是边界。
- 计算长度为 len 的区间时，依赖的都是更短区间。

5.1 思想

区间 DP 通常处理“合并一段区间”或“删除一段区间”的问题。

常见状态：

$$f_{l,r}$$

表示只考虑区间 $[l, r]$ 的答案。

区间 DP 的典型转移是枚举分割点：

$$f_{l,r} = \min_{l \leq k < r} \{f_{l,k} + f_{k+1,r} + \text{cost}(l, r)\}$$

计算顺序一般按区间长度从小到大：

- 长度为 1 的区间是边界。
- 计算长度为 len 的区间时，依赖的都是更短区间。

重点：区间 DP 的顺序通常是“长度 -> 左端点 -> 分割点”。

5.2 石子合并

5.2 石子合并

- P1880 [NOI1995] 石子合并

有 n 堆石子排成一圈，每次可以合并相邻两堆，代价为两堆石子总数。求最小和最大总代价。

5.2 石子合并

- P1880 [NOI1995] 石子合并

有 n 堆石子排成一圈，每次可以合并相邻两堆，代价为两堆石子总数。求最小和最大总代价。

先考虑链上的版本。

设 $f_{l,r}$ 表示将 $[l, r]$ 合并成一堆的最小代价。

最后一次合并一定把 $[l, r]$ 分成两段 $[l, k]$ 和 $[k + 1, r]$:

$$f_{l,r} = \min_{l \leq k < r} \{ f_{l,k} + f_{k+1,r} + \sum(l, r) \}$$

5.2 石子合并

- P1880 [NOI1995] 石子合并

有 n 堆石子排成一圈，每次可以合并相邻两堆，代价为两堆石子总数。求最小和最大总代价。

先考虑链上的版本。

设 $f_{l,r}$ 表示将 $[l, r]$ 合并成一堆的最小代价。

最后一次合并一定把 $[l, r]$ 分成两段 $[l, k]$ 和 $[k + 1, r]$ ：

$$f_{l,r} = \min_{l \leq k < r} \{ f_{l,k} + f_{k+1,r} + \sum(l, r) \}$$

其中 $\sum(l, r)$ 可用前缀和 $O(1)$ 求出。

边界： $f_{i,i} = 0$ ，单独一堆不需要合并。

5.2 石子合并

- P1880 [NOI1995] 石子合并

有 n 堆石子排成一圈，每次可以合并相邻两堆，代价为两堆石子总数。求最小和最大总代价。

先考虑链上的版本。

设 $f_{l,r}$ 表示将 $[l, r]$ 合并成一堆的最小代价。

最后一次合并一定把 $[l, r]$ 分成两段 $[l, k]$ 和 $[k + 1, r]$:

$$f_{l,r} = \min_{l \leq k < r} \{ f_{l,k} + f_{k+1,r} + \sum(l, r) \}$$

其中 $\sum(l, r)$ 可用前缀和 $O(1)$ 求出。

边界: $f_{i,i} = 0$, 单独一堆不需要合并。

环形问题常见处理：把序列复制一遍，在长度为 n 的所有区间中取答案。

5.3 能量项链

5.3 能量项链

- P1063 [NOIP2006 提高组] 能量项链

有一圈珠子，合并相邻珠子会产生能量，求最大总能量。

5.3 能量项链

- P1063 [NOIP2006 提高组] 能量项链

有一圈珠子，合并相邻珠子会产生能量，求最大总能量。

这题和石子合并类似，也是环形区间 DP。

设 $f_{l,r}$ 表示合并开区间或闭区间的一段珠子能得到的最大能量，枚举最后一次合并的位置。

5.3 能量项链

- P1063 [NOIP2006 提高组] 能量项链

有一圈珠子，合并相邻珠子会产生能量，求最大总能量。

这题和石子合并类似，也是环形区间 DP。

设 $f_{l,r}$ 表示合并开区间或闭区间的一段珠子能得到的最大能量，枚举最后一次合并的位置。

很多区间 DP 的难点在于：

- 状态区间端点是否包含边界元素。
- 最后一次操作是什么。
- 环形结构如何断开成链。

5.3 能量项链

- P1063 [NOIP2006 提高组] 能量项链

有一圈珠子，合并相邻珠子会产生能量，求最大总能量。

这题和石子合并类似，也是环形区间 DP。

设 $f_{l,r}$ 表示合并开区间或闭区间的一段珠子能得到的最大能量，枚举最后一次合并的位置。

很多区间 DP 的难点在于：

- 状态区间端点是否包含边界元素。
- 最后一次操作是什么。
- 环形结构如何断开成链。

重点：区间 DP 常常从“最后一次操作”反推转移。

5.4 删除类区间 DP

5.4 删除类区间 DP

- CF607B Zuma

给定一个序列，每次可以删除一段回文连续子序列，求最少删除次数。

5.4 删除类区间 DP

- CF607B Zuma

给定一个序列，每次可以删除一段回文连续子序列，求最少删除次数。

设 $f_{l,r}$ 表示删除 $[l,r]$ 的最少次数。

基本想法：

- 可以先删除 a_l ，再删除 $[l+1,r]$ 。
- 若 $a_l = a_k$ ，可以尝试让 a_l 和 a_k 在同一次回文删除中被消掉。

5.4 删除类区间 DP

- CF607B Zuma

给定一个序列，每次可以删除一段回文连续子序列，求最少删除次数。

设 $f_{l,r}$ 表示删除 $[l, r]$ 的最少次数。

基本想法：

- 可以先删除 a_l ，再删除 $[l + 1, r]$ 。
- 若 $a_l = a_k$ ，可以尝试让 a_l 和 a_k 在同一次回文删除中被消掉。

这类题不一定是简单的“左右合并”，但依然符合区间 DP 的结构：小区间推出大区间。

5.5 习题

- P1880 [NOI1995] 石子合并
- P1063 [NOIP2006 提高组] 能量项链
- P4170 [CQOI2007] 涂色
- P3205 [HNOI2010] 合唱队
- CF607B Zuma
- CF1132F Clear the String

Thanks!